LAPORAN PRAKTIKUM WORKSHOP PEMROGRAMAN WEB

MODUL: INTEGRASI WHATSAPP GATEWAY (WAHA DOCKER) & EXTERNAL WATCHDOG MONITORING

Mata Kuliah	: Workshop Pemrograman Web (Semester 2)	**Tanggal Uji**	: November 2026
Nama Mahasiswa	: Tegar Mahardika	**No. WhatsApp**	: +628973656200
Repository	: https://github.com/Gaarrr33/Project01_Express	**Status Sistem**	: Terverifikasi (UP & Healthy)

I. TUJUAN PRAKTIKUM

1. **Konfigurasi Self-Hosted WhatsApp Gateway:** Menjalankan container Docker WAHA (WhatsApp HTTP API) Core lokal dengan binding port aman 127.0.0.1:3008 dan persistent storage sesi.
2. **Implementasi Fitur Otentikasi OTP WhatsApp:** Mengintegrasikan modul pengiriman kode OTP 6-digit untuk verifikasi reset password langsung ke akun WhatsApp pengguna.
3. **Penyediaan Endpoint Pemantau Kesehatan:** Menyediakan endpoint GET /health sebagai target probe HTTP yang mengembalikan status respons sistem (HTTP 200).
4. **Pembangunan External Watchdog Monitor:** Mengembangkan daemon monitoring independen (scripts/watchdog.js) yang berjalan terpisah dari proses Express.js dengan interval polling setiap 5 detik.
5. **Mekanisme Alerting Otomatis & Anti-False Alarm:** Mengirim peringatan seketika ke WhatsApp Administrator saat backend down (fail threshold 2x berturut-turut) serta notifikasi pemulihan (resolved) saat server kembali aktif.

II. ARSITEKTUR SISTEM & SPESIFIKASI TEKNIS

Komponen	Peran / Tugas	Konfigurasi / Port
WAHA Core (Docker)	Gateway WhatsApp mandiri berbasis WEBJS engine untuk kirim/terima pesan via REST API.	Container: waha_core Port: 127.0.0.1:3008
Backend Express.js	Aplikasi utama katalog etalase, autentikasi pengguna, route reset password OTP, dan probe /health.	Node.js v18+ Port: 127.0.0.1:3000
External Watchdog	Proses background terpisah untuk mengecek endpoint /health, mendeteksi outage, dan kirim pesan alert WA ke admin.	scripts/watchdog.js Interval: 5s | Threshold: 2x

III. IMPLEMENTASI & KONFIGURASI KODE

1. Konfigurasi Container WAHA Core (docker-compose.yml)

```
services: waha: image: devlikeapro/waha:latest container_name: waha_core restart: unless-stopped ports: - "127.0.0.1:3008:3000" volumes: - ./waha-sessions:/app/.sessions environment: - WHATSAPP_DEFAULT_ENGINE=WEBJS - WHATSAPP_API_KEY=${WAHA_API_KEY}
```

2. Logika Pemantauan External Watchdog (scripts/watchdog.js)

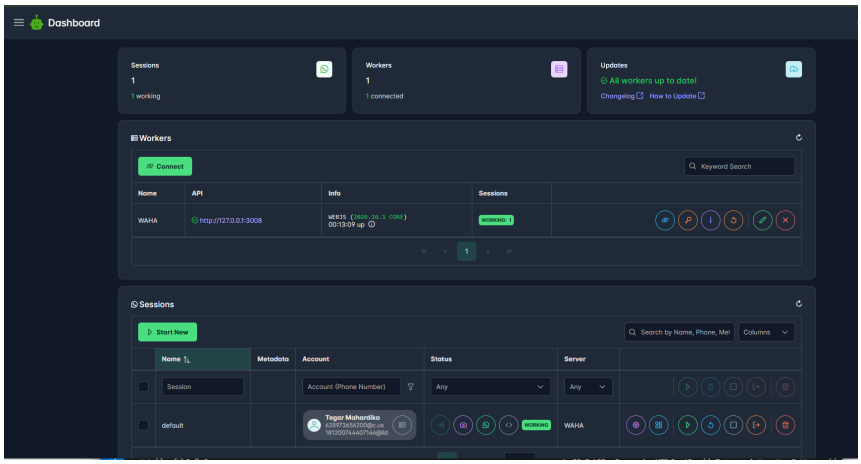
```
const TARGET_URL = 'http://127.0.0.1:3000/health'; const PING_INTERVAL_MS = 5000; const FAIL_THRESHOLD = 2; // Butuh 2x gagal sebelum status DOWN (anti-false alarm) async function checkHealth() { try { const res = await fetch(TARGET_URL, { timeout: 3000 }); if (res.ok) handleSuccess(); else handleFailure(); } catch (err) { handleFailure(); } }
```

IV. HASIL PENGUJIAN DAN ANALISIS DOKUMENTASI

Berikut adalah bukti eksekusi pengujian fungsionalitas sistem yang didokumentasikan secara runtut sesuai tahapan praktikum:

4.1. Dashboard Manajemen Sesi WAHA Core (Port 3008)

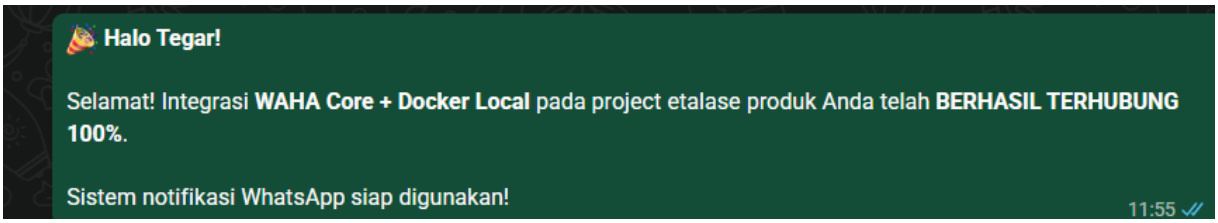
Verifikasi status worker WAHA di browser. Sesi login default terhubung ke nomor Tegar Mahardika (628973656200@c.us) dengan status **WORKING** dan 1 worker aktif melayani permintaan API.



Gambar 1: Dashboard WAHA Core menunjukkan sesi WhatsApp aktif dan siap melayani pengiriman pesan.

4.2. Pengujian Pengiriman Pesan Awal (test-whatsapp.js)

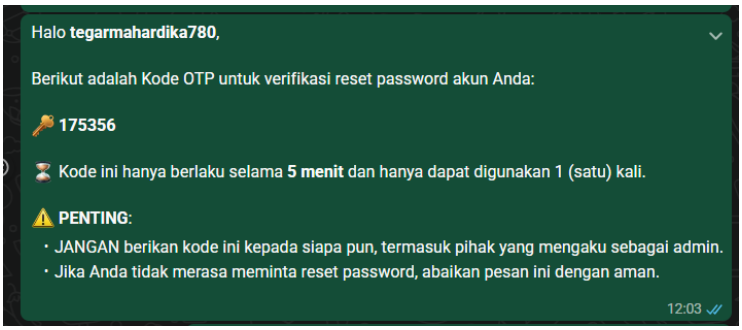
Eksekusi skrip pengujian koneksi untuk memastikan backend dapat memanggil HTTP POST ke WAHA dan pesan berhasil terkirim ke WhatsApp tujuan.



Gambar 2: Notifikasi konfirmasi koneksi WAHA Core + Docker Local berhasil diterima di WhatsApp.

4.3. Implementasi Fitur Kode OTP Reset Password via WhatsApp

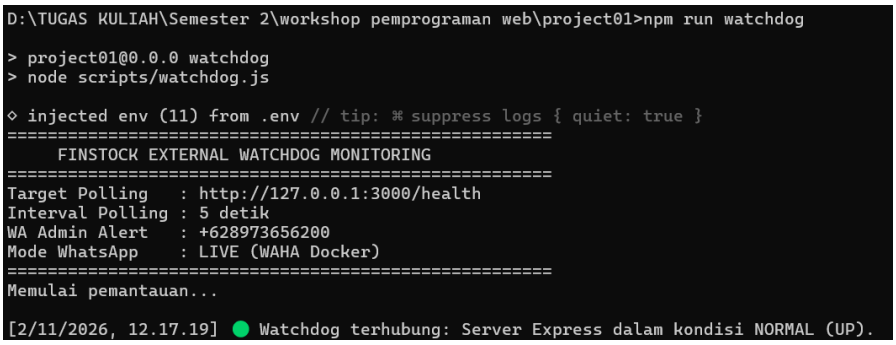
Pengujian alur lupa password pengguna. Sistem men-generate token OTP numerik 6-digit dengan masa aktif 5 menit dan single-use yang dikirimkan secara instan ke nomor WhatsApp terdaftar.



Gambar 3: Format pesan OTP reset password yang diterima oleh pengguna.

4.4. Inisialisasi Monitoring External Watchdog (Kondisi Server Normal / UP)

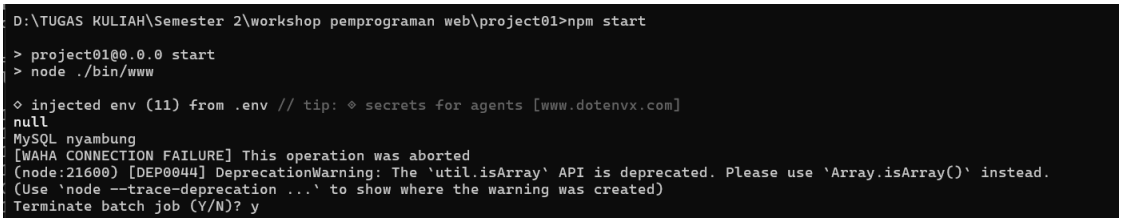
Skrip watchdog.js dijalankan di terminal melalui npm run watchdog. Watchdog membaca konfigurasi .env, mem-polling endpoint http://127.0.0.1:3000/health setiap 5 detik, dan mengonfirmasi bahwa server Express berada dalam kondisi NORMAL (UP).



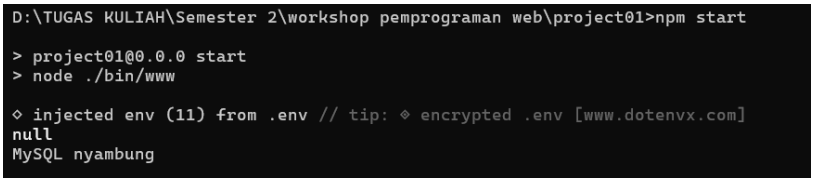
Gambar 4: Terminal watchdog mendeteksi backend Express aktif dalam kondisi normal.

4.5. Siklus Startup Server Backend Express.js

Dokumentasi log saat backend dijalankan via npm start. Pada Gambar 5a terlihat penanganan kegagalan saat container WAHA belum merespons (operation aborted). Pada Gambar 5b backend berhasil berjalan normal dan database MySQL terkoneksi.



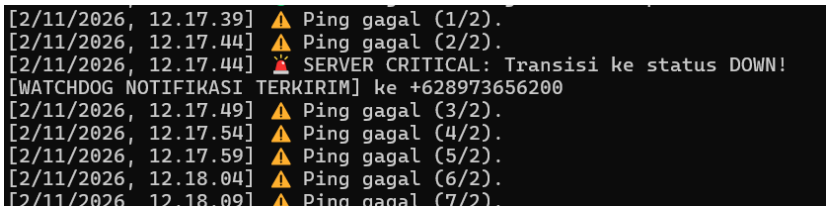
Gambar 5a: Log Express saat percobaan koneksi awal ke WAHA mengalami abort.



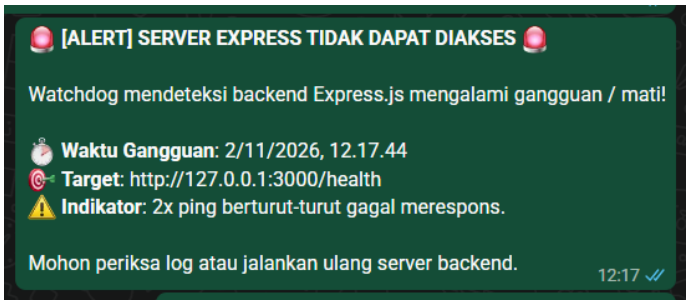
Gambar 5b: Log Express saat server berjalan normal dengan MySQL terkoneksi.

4.6. Simulasi Gangguan Server (Status DOWN & Pengiriman Alert WhatsApp)

Saat backend Express.js dihentikan secara sengaja untuk simulasi gangguan (*crash*): Watchdog mencatat Ping gagal (1/2) pada detik ke-5, lalu Ping gagal (2/2) pada detik ke-10 sehingga ambang batas terpenuhi. Status beralih ke SERVER CRITICAL: DOWN! dan alert langsung dikirim ke WhatsApp Admin.



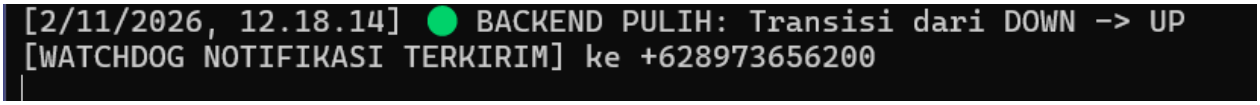
Gambar 6: Terminal Watchdog mendeteksi kegagalan ping beruntun dan memicu alert DOWN.



Gambar 7: Pesan darurat [ALERT] SERVER EXPRESS TIDAK DAPAT DIAKSES diterima di WhatsApp Admin.

4.7. Pemulihan Server (Status UP & Notifikasi RESOLVED)

Saat backend Express.js dinyalakan kembali, probe berikutnya langsung menerima respons HTTP 200 OK. Watchdog melakukan transisi BACKEND PULIH: DOWN -> UP dan secara otomatis mengirimkan pesan pemulihan ke WhatsApp Administrator.



Gambar 8: Terminal Watchdog mendeteksi pemulihan backend Express.js.



Gambar 9: Pesan konfirmasi pemulihan [RESOLVED] SERVER EXPRESS PULIH diterima di WhatsApp Admin.

V. KENDALA DAN PEMECAHAN MASALAH

Kendala / Tantangan	Akar Masalah	Solusi yang Diterapkan
Panggilan API WAHA gagal saat Express baru menyala (Gambar 5a).	Proses booting container WAHA dan inisialisasi browser Chromium memerlukan waktu lebih lama dari startup Express.	Menambahkan mekanisme try-catch dengan timeout 8 detik, serta memastikan health status WAHA valid sebelum request pengiriman.
Potensi spam pesan WhatsApp saat server mati dalam waktu lama.	Jika notifikasi dikirim di setiap ping yang gagal (setiap 5 detik), admin akan menerima ratusan pesan.	Menerapkan State Machine internal pada watchdog: notifikasi hanya dikirim saat terjadi transisi status (UP -> DOWN).
False alarm saat server hanya di-restart sejenak untuk pembaruan kode.	Ping tunggal yang gagal bisa memicu alert darurat meskipun server langsung menyala kembali.	Menetapkan batas FAIL_THRESHOLD = 2, sehingga butuh minimal 2x ping gagal berturut-turut sebelum trigger status DOWN.

VI. KESIMPULAN

1. Integrasi WhatsApp Gateway mandiri menggunakan WAHA Core berbasis Docker berhasil diterapkan secara lokal dengan performa yang stabil dan aman (binding port terbatas pada loopback 127.0.0.1).
2. Fitur pengiriman kode OTP melalui WhatsApp terbukti fungsional dengan masa berlaku ketat (5 menit) untuk mendukung keamanan proses reset password pengguna.
3. Implementasi External Watchdog sebagai proses terpisah terbukti efektif untuk memantau siklus hidup (*lifecycle*) server backend. Pemisahan proses menjamin pemantauan tetap aktif meskipun server aplikasi utama mengalami kegagalan proses (*crash*).
4. Penerapan *fail threshold* (2x kegagalan berturut-turut) dan *state machine* berhasil mengeliminasi potensi alarm palsu (*false alarm*) dan mencegah pengiriman pesan berulang (*spamming*) kepada administrator.

Surabaya, 2 November 2026
Praktikan,

Tegar Mahardika
NIM / Mahasiswa Semester 2